

公交安全管理系统设计

题目

后台数据库

设计公交安全管理系统。数据库的有关语义如下：

1. 公交公司有若干个车队，每个车队下有若干条线路；
2. 公交公司有若干辆汽车，每辆车属于一条线路；
3. 每个车队有一名队长，他只有管理工作，不开车；
4. 每条线路有若干名司机，其中有一名路队长，除开车外，还承担管理工作；每名司机只在一条线路上开车；
5. 司机开车时会产生违章，包含：闯红灯、未礼让斑马线、压线、违章停车等；
6. 队长、路队长负责将司机的违章信息输入到系统，包含：司机、车辆、车队、线路、站点、时间、违章等。

前台程序

开发一个公交安全管理系统来对数据库进行访问，可以使用 Java、Python、C 等集成开发环境。系统实现功能如下：

1. 录入司机基本信息，如工号、姓名、性别等；
2. 录入汽车基本信息，如车牌号、座数等；
3. 录入司机的违章信息；
4. 查询某个车队下的司机基本信息；
5. 查询某名司机在某个时间段的违章详细信息；
6. 查询某个车队在某个时间段的违章统计信息，如：2 次闯红灯、4 次未礼让斑马线等。

注意事项：

1. 在数据库的设计过程中需要运用规范化理论，避免出现插入异常、删除异常、数据冗余等问题；

2. 必须设定关系的完整性规则，如实体完整性（例如设置主码），参照完整性（例如设置外码和对应的主码），用户自定义完整性（例如性别只能为“男”或“女”）；
3. 使用索引来加快查询的速度；使用视图、函数、存储过程来简化系统的设计；

设计过程

需求分析

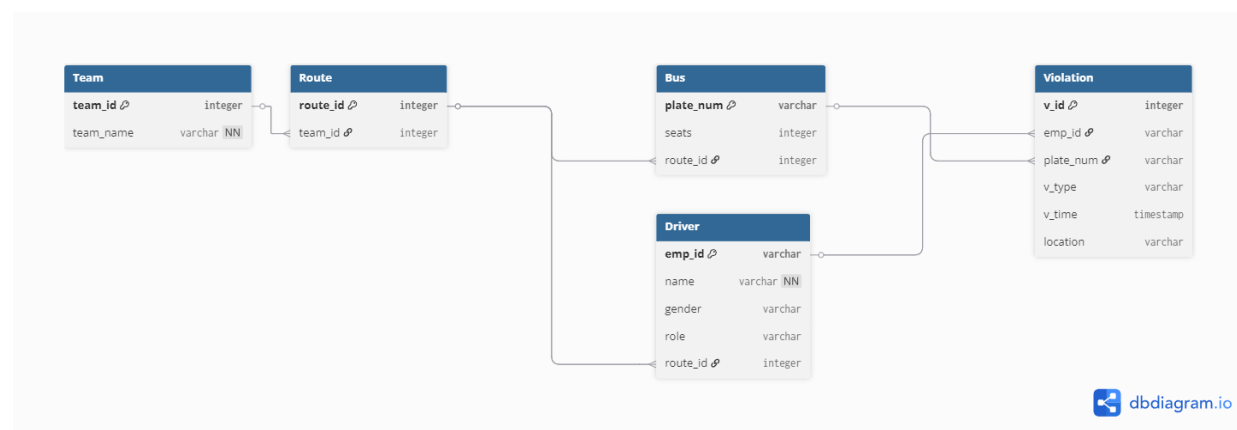
根据提供的业务规则，梳理出以下实体关系：

- 1) 车队与线路：1:N 关系（一个车队管理多条线路）。
- 2) 线路与车辆：1:N 关系（一条线路包含多辆车）。
- 3) 线路与司机：1:N 关系（一名司机只在一条线路上开车）。
- 4) 违章记录：多对多关系的体现（司机、车辆在特定时间发生的事件）。
- 5) 角色区分：队长（仅管理）、路队长（驾驶+管理）、普通司机（驾驶）。

功能需求

- 1) 数据录入：司机、车辆、违章信息。
- 2) 数据查询：按车队查司机、按司机查违章。
- 3) 统计分析：按车队、时间段统计各类违章次数。

概念结构设计 (E-R 图描述)



逻辑结构设计 (关系模型)

1) 车队表 (Team)

team_id (PK, int): 车队编号

team_name (varchar): 车队名称

2) 线路表 (Route)

route_id (PK, int): 线路号

team_id (FK): 所属车队编号

3) 车辆表 (Bus)

plate_num (PK, varchar): 车牌号

seats (int): 座位数

route_id (FK): 所属线路

4) 员工表 (Driver)

emp_id (PK, varchar): 工号

name (varchar): 姓名

gender (char): 性别 (Check: '男' or '女')

role (varchar): 职位 (队长/路队长/司机)

route_id (FK): 所属线路 (队长可为空或特定规则, 此处假设关联线路)

5) 违章记录表 (Violation)

v_id (PK, integer, auto_increment): 记录编号

emp_id (FK): 司机工号

plate_num (FK): 车牌号

v_type (varchar): 违章类型 (闯红灯/压线等)

v_time (datetime): 违章时间

location (varchar): 地点

程序开发环境

操作系统: Windows

开发语言: Python 3.14

IDE: PyCharm Community

数据库: SQLite3

前端 GUI 库: Tkinter

应用程序设计中遇到的问题及解决方法

问题：频繁打开关闭数据库影响性能。

解决：使用全局连接对象或上下文管理器 (Context Manager) 确保连接正确关闭。

问题：统计某车队违章需要关联 4 张表 (车队-线路-司机-违章)，SQL 语句复杂。

解决：使用 SQL JOIN 语句或创建视图 (VIEW) 来简化查询逻辑。

问题：用户输入时间格式不统一。

解决：前端使用简单的格式验证，数据库存储为标准字符串 (YYYY-MM-DD HH:MM:ss)。

总结

本系统完成了从需求分析、数据库设计到应用开发的完整流程。我将课堂上学习的 1NF、2NF、3NF 规范化理论应用于实际项目中，通过合理设计表结构，成功避免了数据冗余和更新异常问题。掌握了从业务需求分析到 ER 图设计的完整流程，能够将复杂的业务逻辑转化为清晰的数据模型。我积累了前后端集成开发经验，体验了完整项目开发流程，提升了问题解决能力，培养了工程实践意识。这次项目经验将成为我未来学习和职业发展的重要基石，我将继续在数据库技术领域深入探索，努力成为一名优秀的数据工程师。

附录

```
import sqlite3                                self.init_db()

import tkinter as tk

from tkinter import ttk, messagebox           def init_db(self):

from datetime import datetime                 """初始化数据库结构，创建表和视

                                              图"""

class Database:                               self.cur.execute("PRAGMA

def __init__(self, db_file="bus_data.db"):    foreign_keys = ON;")

    self.conn = sqlite3.connect(db_file)

    self.cur = self.conn.cursor()             # 1. 车队表
```

```

self.cur.execute("""
CREATE TABLE IF NOT EXISTS Team (
team_id INTEGER
PRIMARY KEY,
team_name TEXT NOT NULL
)
""")

# 2. 线路表
self.cur.execute("""
CREATE TABLE IF NOT EXISTS Route (
route_id INTEGER
PRIMARY KEY,
team_id INTEGER,
FOREIGN KEY (team_id) REFERENCES Team(team_id)
)
""")

# 4. 员工表 (司机/队长)
self.cur.execute("""
CREATE TABLE IF NOT EXISTS Driver (
emp_id TEXT PRIMARY KEY,
name TEXT NOT NULL,
gender TEXT
CHECK(gender IN ('男', '女')),
role TEXT,
route_id INTEGER,
FOREIGN KEY (route_id) REFERENCES Route(route_id)
)
""")

# 5. 违章记录表
self.cur.execute("""
CREATE TABLE IF NOT EXISTS Violation (
v_id INTEGER PRIMARY KEY AUTOINCREMENT,
emp_id TEXT,
plate_num TEXT,
route_id INTEGER,
v_type TEXT,
)
""")

# 3. 车辆表
self.cur.execute("""
CREATE TABLE IF NOT EXISTS Bus (
plate_num TEXT
PRIMARY KEY,
seats INTEGER,
route_id INTEGER,
v_id INTEGER PRIMARY KEY AUTOINCREMENT,
emp_id TEXT,
plate_num TEXT,
v_type TEXT,
)
""")

```

```

        v_time TEXT,
        location TEXT,
        FOREIGN KEY (emp_id)
REFERENCES Driver(emp_id),
        FOREIGN KEY
(plate_num) REFERENCES Bus(plate_num)
    )
    """
    def add_driver(self, emp_id, name, gender,
role, route_id):
        try:
            self.cur.execute("CREATE INDEX IF
NOT EXISTS idx_v_time ON
Violation(v_time);")
            self.cur.execute("INSERT INTO
Driver VALUES ('D001', '张三', '男', '司机',
101)")
            self.cur.execute("INSERT INTO
Driver VALUES ('D002', '李四', '女', '路队长',
201)")
            print("初始化测试数据完成。")
        except:
            pass
    # 创建索引优化查询
    self.cur.execute("CREATE INDEX IF
NOT EXISTS idx_v_time ON
Violation(v_time);")
    # 预制一些基础数据
    self.preload_data()
    self.conn.commit()
    def preload_data(self):
        # 检查是否已有数据，无则插入
        self.cur.execute("SELECT count(*)
FROM Team")
        if self.cur.fetchone()[0] == 0:
            self.cur.execute("INSERT INTO
Team VALUES (1, '一车队'), (2, '二车队')")
            self.cur.execute("INSERT INTO
Route VALUES (101, 1), (102, 1), (201, 2)")
            self.cur.execute("INSERT INTO
Bus VALUES ('京 A88888', 45, 101), ('京
A12345', 30, 201)")
            self.conn.commit()
    def add_bus(self, plate, seats, route_id):
        try:
            self.cur.execute("INSERT INTO
Bus VALUES (?, ?, ?)", (plate, seats, route_id))
            self.conn.commit()
            return True, "添加成功"
        except sqlite3.IntegrityError as e:
            return False, f"添加失败: {e}"
        except sqlite3.Error as e:
            return False, str(e)

```

```

def add_violation(self, emp_id, plate,
v_type, v_time, location):
    try:
        self.cur.execute("INSERT INTO
Violation (emp_id, plate_num, v_type, v_time,
location) VALUES (?, ?, ?, ?, ?)",
(emp_id,
plate, v_type, v_time, location))
        self.conn.commit()
        return True, "录入成功"
    except sqlite3.Error as e:
        return False, str(e)

```

```

def query_drivers_by_team(self, team_id):
    # 多表连接查询：车队 -> 线路 ->
司机
    sql = """
SELECT d.emp_id, d.name, d.gender,
d.role, r.route_id, t.team_name
FROM Driver d
JOIN Route r ON d.route_id =
r.route_id
JOIN Team t ON r.team_id = t.team_id
WHERE t.team_id = ?
"""
    self.cur.execute(sql, (team_id,))
    return self.cur.fetchall()

```

```

def query_violation_by_driver(self, emp_id,
start_date, end_date):

```

```

sql = """
SELECT v.v_time, v.v_type,
v.location, v.plate_num
FROM Violation v
WHERE v.emp_id = ? AND v.v_time
BETWEEN ? AND ?
ORDER BY v.v_time DESC
"""
self.cur.execute(sql, (emp_id,
start_date, end_date))
return self.cur.fetchall()

```

```

def stats_violation_by_team(self, team_id,
start_date, end_date):
    # 统计查询
    sql = """
SELECT v.v_type, COUNT(*)
FROM Violation v
JOIN Driver d ON v.emp_id =
d.emp_id
JOIN Route r ON d.route_id =
r.route_id
WHERE r.team_id = ? AND v.v_time
BETWEEN ? AND ?
GROUP BY v.v_type
"""
    self.cur.execute(sql, (team_id,
start_date, end_date))
    return self.cur.fetchall()

```

```

def __del__(self):
    self.conn.close()

class App:
    def __init__(self, root):
        self.db = Database()
        self.root = root
        self.root.title("公交安全管理系统")
        self.root.geometry("800x600")

        # 选项卡控件
        self.notebook = ttk.Notebook(root)
        self.notebook.pack(expand=True, fill='both')

        # 创建各个页面
        self.create_input_tab()
        self.create_query_tab()
        self.create_stats_tab()

    def create_input_tab(self):
        frame = ttk.Frame(self.notebook)
        self.notebook.add(frame, text="数据录入")

        # --- 区域 1: 录入司机 ---
        lf1 = ttk.LabelFrame(frame, text="录入司机信息")
        lf1.pack(pady=10, padx=10, fill="x")

        self.conn.close()

        tk.Label(lf1, text="工号:").grid(row=0, column=0, padx=5)

        self.e_uid = ttk.Entry(lf1)
        self.e_uid.grid(row=0, column=1)

        tk.Label(lf1, text="姓名:").grid(row=0, column=2, padx=5)

        self.e_name = ttk.Entry(lf1)
        self.e_name.grid(row=0, column=3)

        tk.Label(lf1, text="性别:").grid(row=0, column=4, padx=5)

        self.e_gender = ttk.Combobox(lf1, values=["男", "女"], width=5)
        self.e_gender.grid(row=0, column=5)

        tk.Label(lf1, text="线路ID:").grid(row=1, column=0, padx=5)

        self.e_route = ttk.Entry(lf1)
        self.e_route.grid(row=1, column=1)

        btn_add_driver = ttk.Button(lf1, text="添加司机", command=self.action_add_driver)
        btn_add_driver.grid(row=1, column=4, columnspan=2, pady=5)

        # --- 区域 2: 录入违章 ---

```



```

lf2 = ttk.LabelFrame(frame, text="录入违章信息")

lf2.pack(pady=10, padx=10, fill="x")

ttk.Label(lf2, text="司机工号:").grid(row=0, column=0)

self.v_uid = ttk.Entry(lf2)

self.v_uid.grid(row=0, column=1)

ttk.Label(lf2, text="车牌号:").grid(row=0, column=2)

self.v_plate = ttk.Entry(lf2)

self.v_plate.grid(row=0, column=3)

ttk.Label(lf2, text="违章类型:").grid(row=1, column=0)

self.v_type = ttk.Combobox(lf2, values=["闯红灯", "未礼让斑马线", "压线", "违章停车"])

self.v_type.grid(row=1, column=1)

ttk.Label(lf2, text="时间 (Y-M-D):").grid(row=1, column=2)

self.v_time = ttk.Entry(lf2)

self.v_time.insert(0, datetime.now().strftime("%Y-%m-%d")) # 默认当天

self.v_time.grid(row=1, column=3)

```

```

btn_add_v = ttk.Button(lf2, text="提交违章", command=self.action_add_violation)

btn_add_v.grid(row=2, column=1, pady=10)

def create_query_tab(self):

    frame = ttk.Frame(self.notebook)

    self.notebook.add(frame, text="查询功能")

    # 查询车队司机

    lf1 = ttk.LabelFrame(frame, text="查询车队下的司机")

    lf1.pack(fill="x", padx=10, pady=5)

    ttk.Label(lf1, text="车队 ID (1, 2...):").pack(side="left", padx=5)

    self.q_team_id = ttk.Entry(lf1, width=5)

    self.q_team_id.pack(side="left")

    ttk.Button(lf1, text="查询", command=self.action_query_team).pack(side="left", padx=10)

    self.tree_driver = ttk.Treeview(frame, columns=("ID", "Name", "Gender", "Role", "Route"), show="headings", height=5)

    self.tree_driver.heading("ID", text="工号")

```

```

        self.tree_driver.heading("Name",
text="姓名")

        self.tree_driver.heading("Gender",
text="性别")

        self.tree_driver.heading("Role",
text="职位")

        self.tree_driver.heading("Route",
text="线路")

        self.tree_driver.pack(fill="x",
padx=10)

        # 查询个人违章

        lf2 = ttk.LabelFrame(frame, text="查
询司机违章明细")

        lf2.pack(fill="x", padx=10, pady=5)

        ttk.Label(lf2, text="工
号:").pack(side="left")

        self.q_v_uid = ttk.Entry(lf2, width=10)

        self.q_v_uid.pack(side="left")

        ttk.Button(lf2, text=" 查 询  ( 本 月 )",
command=self.action_query_driver_v).pack(sid
e="left", padx=10)

        self.tree_v = ttk.Treeview(frame,
columns=("Time", "Type", "Loc", "Plate"),
show="headings", height=5)

        self.tree_v.heading("Time", text=" 时
间")

        self.tree_v.heading("Type", text=" 违
章类型")

        self.tree_v.heading("Loc", text=" 地 点
")

        self.tree_v.heading("Plate", text=" 车
牌")

        self.tree_v.pack(fill="x", padx=10)

        def create_stats_tab(self):

            frame = ttk.Frame(self.notebook)

            self.notebook.add(frame, text=" 统 计
报表")

            lf = ttk.LabelFrame(frame, text="车队
违章统计")

            lf.pack(pady=10)

            ttk.Label(lf, text=" 车 队
ID:").grid(row=0, column=0)

            self.s_team = ttk.Entry(lf, width=5)

            self.s_team.grid(row=0, column=1)

            ttk.Button(lf, text=" 生 成 统 计 ",
command=self.action_stats).grid(row=0,
column=2, padx=10)

            self.txt_stats = tk.Text(frame,
height=10)

            self.txt_stats.pack(padx=10, pady=10,
fill="both")

        # --- 事件处理函数 ---

```

```

        self.tree_driver.insert("", "end",
                                values=r)

    def action_add_driver(self):
        # 简单获取数据，实际需更严格校验
        success, msg = self.db.add_driver(self.e_uid.get(),
        self.e_name.get(),
        self.e_gender.get(), "司机", self.e_route.get())
        messagebox.showinfo("结果", msg)

    def action_add_violation(self):
        success, msg = self.db.add_violation(self.v_uid.get(),
        self.v_plate.get(),
        self.v_type.get(), self.v_time.get(), "市区道路")
        messagebox.showinfo("结果", msg)

    def action_query_team(self):
        # 清空表格
        for row in self.tree_driver.get_children():
            self.tree_driver.delete(row)

        tid = self.q_team_id.get()
        if not tid: return

        res = self.db.query_drivers_by_team(tid)
        for r in res:

self.tree_driver.insert("", "end",
                        values=r)

    def action_query_driver_v(self):
        for row in self.tree_v.get_children():
            self.tree_v.delete(row)

        uid = self.q_v_uid.get()
        # 简化处理，默认查询 2020 年到 2030 年
        res = self.db.query_violation_by_driver(uid, "2020-01-01", "2030-12-31")
        for r in res:
            self.tree_v.insert("", "end",
                                values=r)

    def action_stats(self):
        tid = self.s_team.get()
        if not tid: return

        res = self.db.stats_violation_by_team(tid, "2020-01-01", "2030-12-31")

        self.txt_stats.delete(1.0, "end")
        self.txt_stats.insert("end", f"车队 {tid} 违章统计 (2020-2030):\n")
        self.txt_stats.insert("end", "-"*30 + "\n")

        if not res:

```

```
self.txt_stats.insert("end", "无记  
录")  
  
for r in res:  
    self.txt_stats.insert("end", f" 类  
型: {r[0]} \t 次数: {r[1]}\n")  
  
if __name__ == "__main__":  
    root = tk.Tk()  
    app = App(root)  
    root.mainloop()
```

效果演示在视频中

公交安全管理系统

—□×

数据录入 查询功能 统计报表

录入司机信息

工号:

姓名:

性别:

▼

线路ID:

添加司机

录入违章信息

司机工号:

车牌号:

违章类型:

▼

时间(Y-M-D):

2025-12-28

提交违章